TPE: TTX: Practical Triton Autotuning via Learned Latency Prediction

Abstract

GPU kernel optimization is essential for high-performance machine learning systems, but it remains labor-intensive. Triton simplifies GPU kernel development with a Python-based interface, yet autotuning still requires searching a large configuration space, which is costly. We present TTX, a Triton tuning framework with an XGBoost-based performance predictor. By using input shapes, tuning parameters, and IR-level features, TTX accurately estimates kernel latency and quickly selects promising candidates for further search. Experiments show that TTX achieves around 10% MAPE on operators such as matrix multiplication, batched matrix multiplication, and convolution across multiple GPUs. It reaches about 80% of the best performance with the top-1 candidate and over 95% with the top-50 candidates. We further analyze cross-kernel generalization and identify a fundamental asymmetry — pointwise and normalization kernels transfer cleanly across the kernel boundary while compute-bound kernels do not — empirically motivating the per-kernel design.

Keywords

GPU kernel, performance modeling, Triton, autotuning

ACM Reference Format:

. 2026. TPE: TTX: Practical Triton Autotuning via Learned Latency Prediction. In . ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Deep learning (DL) workloads have demonstrated great success in various problem domains, such as computer vision [1–3] and natural language processing [4–6]. However, they typically involve massive general matrix multiplication (GEMM)-based operators, such as CONV2D, Linear, and BMM (batched matrix multiplication), complex non-GEMM operators, such as memory operations (e.g., reshape, view, and transpose), normalization, and logit computation functions (e.g., Softmax), and/or sophisticated compound kernels such as FlashAttention [7, 8] and Mixture-of-Experts (MoE) [9, 10]. The critical operations, such as attention, can also span into multiple implementations for prefill and decode phases [11]. To effectively support these diverse operations, modern hardware systems [12–16] have involved scaling up the number of computing units, cache size, memory capacity, and bandwidth. This motivates a strong demand to develop and optimize GPU kernels and DL frameworks, such as Tensorflow [17], Pytorch [18, 19], vLLM [20], and SGLang [21].

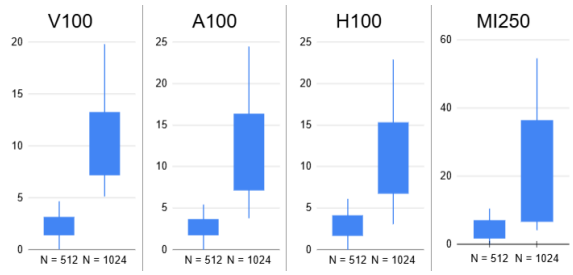


Figure 1: Normalized latencies of a Triton matrix multiplication across configurations on Nvidia V100, A100, H100, and AMD MI250. The input sizes are $(M, N, K) = (N, N, N)$ with $N \in \{512, 1024\}$. The Y-axis represents latency normalized by the minimum one at $N = 512$.

To facilitate kernel development, various domain-specific languages (DSL) and their corresponding platforms have recently been introduced, including TVM [22, 23], Halide [24], and Triton [25]. Specifically, Triton [25] has emerged as a programming language and compiler that enables developers to write high-performance GPU kernels with reduced programming complexity. For example, a numerical evaluation of Triton that demonstrates its ability to generate matrix multiplication implementations on par with cuBLAS and up to 3× faster than other domain-specific languages for DNNs on recurrent and transformer neural networks [25]. As a result, Triton has been integrated into various DL platforms, including TorchInductor [19], vLLM [20], and SGLang [21], attracting significant attention from the developer community.

A critical issue with a Triton kernel is that it typically requires tuning hyperparameters to achieve good performance [26]. For example, Figure 1 shows a normalized latency of a matrix multiplication kernel in Triton on four common GPUs (e.g., V100 [12], A100 [14], H100, and MI250 [15]). These kernel implementations are associated with different configurations, including tiling configurations and thread block dimensions. It is observed that latencies vary widely across configurations, for example, ranging from 1 to 16× with an input shape of $(1024, 1024, 1024)$ (See Section 4 for detailed settings). Unfortunately, it becomes too costly to fully examine all tuning configurations (e.g., with a few thousand candidates) to identify the best-performing one. According to a recent report [26], among 57 Triton kernels in vLLM [20], only seven have a performance-tuning mode by default.

To address these shortcomings, this work presents TTX, a tuning scheme for Triton kernels with an XGBoost performance predictor, with the following main contributions:

- **Autotuner.** We present TTX, an extension of the Triton-JIT compiler that transparently and efficiently supports performance autotuning for existing applications. Leveraging an efficient latency predictor, TTX rapidly explores the tuning

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

configuration space and thereby accurately identifies a few potential configuration candidates (e.g., the top-50 via a performance model) for benchmarking. [1]

- **Performance Model.** The proposed predictor is built on a rich set of features specified for Triton kernels. Leveraging a well-known regressor, XGBoost [27], our predictor rapidly and accurately estimates the latency of a Triton kernel for a given tuning configuration.
- **Evaluation.** We evaluate TTX on a comprehensive set of Triton operations, including emerging operations such as FlashAttention, quantized GEMM, and fused MoE. Experiments are also conducted on four popular GPUs (e.g., Nvidia V100, A100, and H100, and AMD MI250).
- **Cross-Kernel Generalization Analysis**. We empirically study leave-one-kernel-out (LOKO) generalization with a unified cross-kernel predictor that combines a contrastive-pretrained PTX encoder with a single shared XGBoost. The analysis reveals that pointwise and normalization kernels transfer cleanly across the kernel boundary, while compute-bound kernels (MM, BMM) do not. This asymmetry directly motivates the per-kernel design of TTX and clarifies the boundary along which a unified model can and cannot substitute for per-kernel ones.

2 Background

2.1 GPU Kernel Development and Generation

Modern DL workloads typically consist of various complex operations, including MM, BMM, normalization [28, 29], attention [30], and MoE [9]. One popular approach to specializing kernels for various hardware architectures is to use template libraries. For example, `flash_attn` libraries [7, 8] present optimized kernels for widely-used attention operations [30]. These template libraries select which handwritten code fragments (written in a low-level language such as CUDA) to use based on the usage scenario (e.g., depending on the tensor shape, data type, or batch size). Unfortunately, `flash_attn` has nearly 70000 LoC, making it a significant effort to optimize the kernel or modify it to target a new hardware device.

The DSL Triton [25] has recently become a promising alternative to writing custom CUDA kernels. Triton enables developers to write and debug Python-based code that can be executed on various GPUs, including Nvidia V100 [12], A100 [14], H100, and AMD MI250 [15]. For example, Figure 2 presents an example of Triton matrix multiplication (`triton_mm`) in Lines 25-38. Written in Python, the code takes as input a set of parameters, including input shapes, tile sizes, and parallelism factors. Thanks to a user-friendly interface, Triton has been integrated into popular DL platforms, such as Torch Inductor [19, 31] and vLLM [20], to generate efficient GPU kernels. For instance, given a PyTorch layer normalization implementation, Torch Inductor can generate a Triton code template, enabling the rapid development of an efficient kernel.

2.2 Compilation and Autotuning

To avoid writing hand-optimized code, one may rely on either ahead-of-time compilation to generate generic binaries or use a Just-in-Time (JIT) compiler to generate executables on the fly [26]. Compiler-based approaches depend on predefined heuristics to generate general-purpose code, which may not maximize performance for all parameter combinations. In contrast, autotuned kernels augment compilation with empirical performance tuning, generating and benchmarking a wide range of kernel variants to select the best-performing configuration for the target hardware and scenario [26]. Specifically, Triton leverages a JIT compiler and explores the tuning configuration design space using hierarchical tiles to automate memory coalescing, shared memory allocation, and thread synchronization [25]. For instance, the autotune function `triton.autotune` (Lines 15-23 in Figure 2) calls a function (e.g., `matmul_get_configs`) to obtain tuning configurations (e.g., BLOCK_M, BLOCK_N, and BLOCK_K).

This infrastructure is notably adopted by the torch.compile API, which leverages the TorchInductor backend to generate Triton kernels [19]. To optimize these kernels, TorchInductor offers a `max-autotune`

Figure 2: A Triton matrix multiplication kernel with top-k autotuning.

```
1  def matmul_get_configs():
2      return [
3          triton.Config(
4              {
5                  'BLOCK_M': BM,
6                  'BLOCK_N': BN,
7                  'BLOCK_K': BK,
8                  'GROUP_M': GM
9              },
10             num_stages=s,
11             num_warps=w
12         )
13     ]
14
15 @triton.autotune(
16     configs=matmul_get_configs(),
17     key=["M", "N", "K"],
18     prune_configs_by={
19         "perf_model": None,
20         "top_k": 50,
21         "early_config_prune": None,
22     },
23     cache_results=True,
24 )
25 @triton.jit
26 def triton_mm(
27     a_ptr, b_ptr, c_ptr,
28     M, N, K,
29     stride_am, stride_ak,
30     stride_bk, stride_bn,
31     stride_cm, stride_cn,
32     BLOCK_M: tl.constexpr,
33     BLOCK_N: tl.constexpr,
34     BLOCK_K: tl.constexpr,
35     GROUP_M: tl.constexpr,
36 ):
37     # kernel body
38     ...
39
40 def perf_model(configs, named_args, **kwargs):
41     return np.array(preds, dtype=np.float32)
42
43 def early_config_prune(configs, named_args, **kwargs):
44     return pruned
45
46 triton_mm.prune_configs_by["perf_model"] = perf_model
47 triton_mm.prune_configs_by["early_config_prune"] =
       early_config_prune
```

[1] Repository, documentation, pretrained models, and benchmarks are anonymized for double-blind review; the public URL will be provided in the camera-ready version.

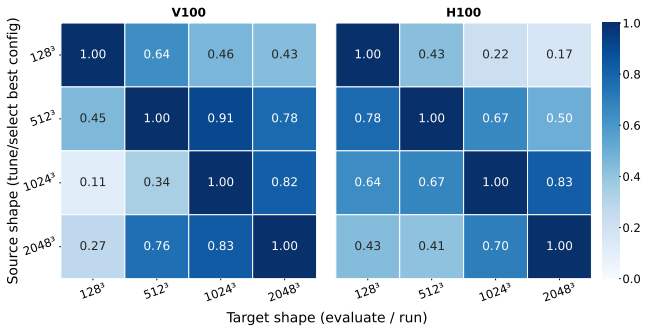


Figure 3: Normalized latency of a Triton GEMM kernel configurations transferred between input shapes ($M = N = K = L$, shown as L^3). Entry (i, j) reports the latency of the configuration tuned on source shape i and evaluated on target j, normalized by the best (lowest-latency) configuration on target j, for Nvidia V100 and H100 GPUs.

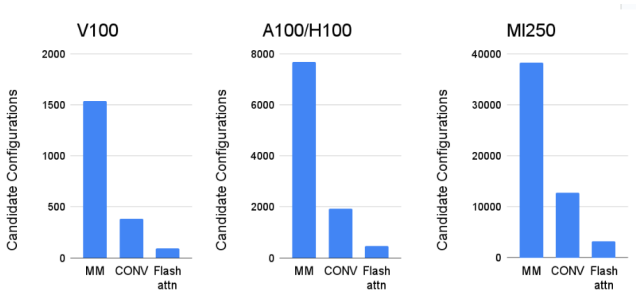


Figure 4: The number of configurations for MM, CONV, and FlashAttention Triton kernels across GPUs.

mode. This mode leverages Triton or template-based matrix multiplications on supported devices and Triton-based convolutions on GPUs, instantiating a benchmarking harness that compiles and executes multiple candidate kernels, varying parameters such as tile sizes and thread parallelism (e.g., `num_warps`) to empirically select the fastest implementation from a static, engineer-defined subset of configurations. While this strategy balances compilation time with runtime performance, finding a "good enough" kernel, it remains fundamentally constrained by its search space. By iterating only over predefined candidates, the compiler may fail to discover optimal configurations that lie outside these heuristics, particularly for irregular input shapes or novel hardware architectures where manual presets are insufficient.

2.3 GPU Kernel Performance Modeling

To evaluate an implementation candidate, one may rely on direct measurements on a device or on a cost model. Measuring performance with all scenarios is generally expensive. As a result, GPU kernel modeling has been well studied in the literature. A common approach relies on analytical models based on handcrafted rules and architectural knowledge [32–35]. They typically rely on runtime profiling and hardware-specific counters such as cache hit rates and memory utilization. Recently, data-driven approaches have emerged as a modern solution, achieving higher accuracy in performance prediction, as demonstrated by systems such as Neusight [36] and by Li et al. [37]. Neusight and Li et al. develop data-driven approaches for cuDNN, demonstrating that predicting kernel latency in this context is relatively straightforward.

3 Proposed TTX

3.1 Motivation

Although tuning Triton kernels is essential to achieve good performance, it poses the following two challenges.

Performance Heterogeneity of Triton Kernels. The latency diversity of Triton kernels is shown in Figure 1. Furthermore, we observe that simply copying the best configuration from one input shape to another can significantly degrade performance. Figure 3 presents a confusion matrix of normalized latency when transferring tuning configurations of Triton-based matrix multiplication among four input shapes. For example, on H100, when using the best configuration with the input shape $(128, 128, 128)$, the implementations of the shapes $(512, 512, 512)$, $(1024, 1024, 1024)$, and $(2048, 2048, 2048)$ only achieve 43%, 22%, and 17%, respectively, compared to their corresponding best-performing implementations. Additionally, transferring the best-performing configuration from one GPU to another (e.g., from A100 to MI250) may also degrade performance significantly [26].

Large Design Space of Configurations. Another challenge in tuning a Triton kernel is the high cost of evaluating the large number of tuning configurations. For example, Figure 4 presents an example of a configuration search space for various operations (e.g., matrix multiplication (MM), convolution (CONV), and flash attention (flash_attn)) on V100, A100/H100, and MI250 (See Section 4 for detailed settings). On Nvidia A100 and H100 GPUs, the widely-used MM may have 7680 configurations, representing all combinations of tile sizes, the number of wraps, and the number of compilation stages. The number of options can be significantly increased on an AMD MI250 GPU (e.g., 38400 configurations). Note that the number of configurations is defined for a single input shape, e.g., $M = 128, N = 128, K = 128$. As explained in the previous example, it may not directly transfer the optimized configuration from one input shape to another. As it becomes extremely costly or infeasible to examine all configurations for all input shapes. Some kernels, such as FlashAttention, may use a small set of default tuning configurations [7, 8].

3.2 System Overview

To address the aforementioned challenges, we introduce TTX, a tuning scheme for Triton kernels with an XGBoost performance predictor. The three building blocks of TTX are explained as follows.

3.2.1 An Extension of Triton-JIT Compiler. We present our framework as an extension of the Triton-JIT compiler, enabling transparent and efficient performance autotuning for existing applications. Triton exposes a rich set of performance-tuning parameters, as they are primarily defined by programmers rather than limited to predefined optimization strategies. As explained in Figure 5, our framework extends the Triton language: developers need only specify the parameters to tune, along with optional tuning ranges. The

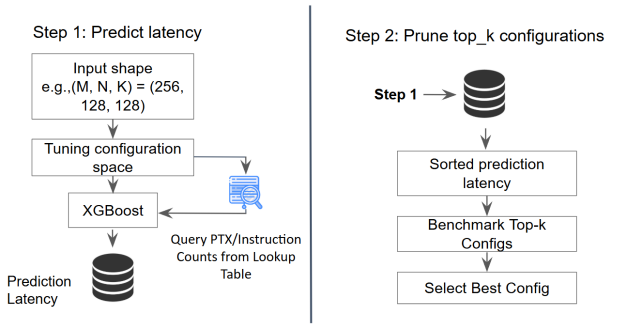


Figure 5: The proposed TTX framework for autotuning Triton kernels.

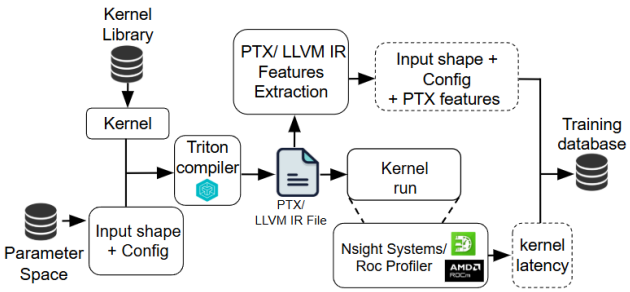


Figure 6: The overall dataset construction flow.

extension comprises a kernel performance model that predicts the execution time of any Triton kernel, coupled with a simple algorithm that selects the best parameters based on the estimated performance.

3.2.2 Kernel Performance Prediction. We build a performance predictor to estimate the execution time of a Triton kernel. The predictor is constructed if a new kernel is encountered (*i.e.*, when it is not already present in our database). We first define the search space of tuning configurations and input shapes. Each set of tuning parameters and input shapes is associated with the corresponding execution time of the kernel, forming a supervised learning problem. Specifically, the kernel performance predictor takes the tuning parameters and the input shape features as input and produces an estimated execution time. During deployment, the predictor generalizes to unseen input shapes and tuning configurations.

To further enhance prediction quality, we also employ intermediate representation (IR) features, which have been shown to be effective [38, 39]. For NVIDIA GPUs, we use PTX, and for AMD GPUs, we use LLVM IR. As illustrated in Fig. 6, each sampled configuration is compiled by the Triton compiler. From the IR, we extract instruction-level features (e.g., instruction counts), as described in Table 1. The extracted code features, together with the corresponding input shapes and tuning parameters, are stored in a database. Finally, we use NVIDIA Nsight Systems and ROCProfiler to measure the runtime latency of each compiled kernel and record these latency measurements alongside the associated input shapes and tuning parameters.

Table 1: Summary of feature groups used for GPU kernel modeling.

Operator-Specific Features (Input Shapes & Tuning)
MM / BMM: $M, N, K, (B)$; `BLOCK_M/N/K`, `GROUP_M`; `div_M/N/K/B`.
Flash_attn: B, S_q, S_k, H, D; `BLOCK_M/N`; `div_indicators` (16-byte alignment).
CONV: N, H, W, C_{in}, C_{out}; `BLOCK_M/N/K`; `div_N/C/H/Cout`.
Pointwise: B, H; `XBLOCK`; `div_B/H` (ReLU, SiLU, GeLU, Softmax, Dropout, LN).
Batch norm: N, C; `BLOCK_C`; `div_N/C`.
Common Architecture & Runtime Features
Execution: `total_blocks`, `threads_per_block`, `waves_per_eu`, `arith_inten`.
PTX/IR: Counts for `total instruction`, `Arith`, `Logic`, `Control`, `Sync`, `Special` ops; `Global/Shared` instruction.

3.2.3 Model-Guided Tuning. As illustrated in Figure 5, our system takes input shapes and candidate tuning parameters, such as tiling sizes, extracts IR-level code features, and uses a trained XGBoost model to predict the kernel execution time. Based on these predictions, our framework selects the top-50 configurations with the lowest estimated latencies. The selected configurations are then executed to measure actual execution times, enabling fine-grained discrimination. The configuration with the fastest execution time is marked. Finally, the kernel is executed with the marked configuration.

3.3 Implementation Details

3.3.1 Triton Kernel Collection. We select a set of representative Triton kernels to evaluate the effectiveness of our framework. The Triton kernels for widely used ML operations are collected from two primary sources: the official Triton repository and kernels generated by TorchInductor. The detailed sources are listed in Table 2. For instance, matrix multiplication, layer normalization, and softmax operations commonly used in large language models (LLMs) are obtained from the Triton repository. TorchInductor, which serves as the default backend for PyTorch's torch.compile [40], produces Triton kernels specialized for particular problem sizes. To enable autotuning and model training, these kernels are restructured to accept parameterized tile sizes and warps, making them tunable across different configurations. The correctness of GPU kernels is validated by comparing their outputs against those of the corresponding PyTorch operators. In addition, kernels for FlashAttention and GEMM quantization are taken from the repositories by the authors, as specified in Table 2.

3.3.2 Performance Features. The feature set employed in our predictive model is categorized into three primary domains: (1) kernel input shapes, (2) tuning parameters, and (3) IR-level code features derived from the generated PTX or LLVM IR code. These features are described in Table 1. Specifically, for the third category, we parse the compiled kernel to extract instruction counts across seven distinct groups: (i) total instructions; (ii) arithmetic operations (e.g., add, sub, cmp); (iii) bitwise logic operations; (iv) special mathematical functions (e.g., sqrt, sin); (v) synchronization primitives; (vi) global memory instructions; and (vii) shared memory instructions. These IR-level code features play an important role in capturing how high-level codes are transformed into lower-level GPU-dependent codes. The IR information helps the model quantitatively identify the portion of each instruction type so that the effect of each tuning parameter is taken into account.

Furthermore, to capture hardware utilization and algorithmic characteristics, we incorporate derived high-level features. These include Arithmetic Intensity, defined as the ratio of total computational operations to the memory footprint of input and output tensors, and Grid Size, representing the total number of thread blocks spawned for the kernel execution.

Our empirical observations suggest that, to ensure efficient hardware memory alignment, the Triton compiler may apply different code generation strategies based on whether tensor dimensions are divisible by 16. To expose this latent behavior to the predictor, we introduce binary alignment indicators (0 or 1) for each dimension. These features aim to assist the model in distinguishing performance patterns between aligned and unaligned input shapes, particularly when fixed tuning configurations are used.

Although tuning parameters and input shapes are both kernel inputs, they are different in nature. Tuning parameters are typically declared with `tl.constexpr` to enable compile-time specialization that reduces computation and improve the compiler's optimization opportunities. As a result, different values of tuning parameters generally produce distinct IR code features. In contrast, for all possible combinations of input shapes, the divisibility-by-16 condition described earlier bounds number of possible uniquely generated IR programs to 2^N, where N denotes the number of input tensors. This property leads to two important implications. First, the number of distinct IR programs generated by a kernel is at most the number of possible tuning configurations, and is independent of the number of input shape combinations, which can be prohibitive large. Second, IR code features can be efficiently generated during the model construction phase by exhaustively iterating through the tuning-parameter space and subsequently reused during the deployment phase.

3.3.3 Prediction Model. As the features are in a tabular structure mixing tile sizes, problem sizes, and the PTX-based counts, our framework adopts XGBoost [27] as a prediction model. XGBoost is chosen for its robustness in regression problems and its ability to efficiently handle nonlinear relationships. Also, in order to be useful in solving real-world compiler problems (*e.g.*, auto-tuning), one of the main concerns when designing our predictor is that the model needs to be light-weight in both training and inference cost. Compared to neural-network-based approaches [36, 37], XGBoost requires a smaller dataset and takes less training time, and thereby aligns well with our design goals.

4 Evaluation

4.1 Methodology

Triton Kernels and GPUs. To demonstrate the effectiveness of the proposed TTX, we conduct the experiments with diverse widely-used DL operations, including matrix multiplication (MM), batched matrix multiplication (BMM), convolution (CONV), activations (e.g., SilU, ReLU, and GeLU), Dropout, Softmax, Batch Normalization (BN), Layer Normalization (LN), quantized GEMM, Flash Attention, and Fused MoE. The detailed sources are specified in Table 2.

The experiments are conducted on four popular data center GPUs, including three NVIDIA models (Tesla V100S-PCIE-32GB, Ampere A100-PCIe-40GB, and H100-HBM3-80GB) and one AMD model (Instinct MI250). All measurements are conducted using Pytorch 2.0.1 compiled with CUDA Library 12.1 for the Nvidia GPUs and Pytorch 2.3.1 compiled with ROCm 6.1 for the AMD GPU. By default, fused MoE and FlashAttention kernels are executed in half-precision (FP16), while the others are executed in FP32. Quantized GEMM kernels are executed using both FP32 and FP16 formats. Latency is measured by running each operator five times and averaging the results from the four later measurements. For training, we collect ~20k data points for each kernel by randomly sampling input shapes and configurations within fixed bounds. Specifically, for input shapes, we uniformly sample: (i) MM/GEMM-Quant: $M, N, K \in [1, 2048]$; (ii) Conv: batch $\in [1, 100]$, channels $\in [3, 512]$, spatial size $\in [5, 1024]$ with a fixed 3×3 kernel, stride 1, padding 1; (iii) BMM: batch $\in [1, 128]$ and $M, N, K \in [1, 2048]$; (iv) FlashAttention: batch $\in [1, 128]$, $L_q, L_k \in [1, 512]$, heads $\in [1, 128]$, head dimension $\in [1, 128]$; (v) MoE: experts $E \in [4, 128]$, top-k $k \in [1, \lfloor E/4 \rfloor]$, tokens $\in [128, 2048]$, out_dim $\in [1025, 3000]$, and in_dim $\in [1024, \text{out\_dim}]$. For all other kernels, any remaining dimensions are sampled from $[1, 2048]$.

Table 2: Collected Triton kernels with abbreviations and upstream sources.

Operator	Short name	Source
Matrix Multiplication	MM	Triton[a]
Batched Matmul	BMM	TorchInductor[b]
Convolution	CONV	TorchInductor[b]
Batch Normalization	BN	TorchInductor[b]
Layer Normalization	LN	Triton[a]
SiLU	SILU	TorchInductor[b]
ReLU	RELU	TorchInductor[b]
GELU	GELU	TorchInductor[b]
Softmax	Softmax	TorchInductor[b]
Dropout	DROPOUT	TorchInductor[b]
FlashAttention V2	Flash_attn	Dao-AILab[c]
Fused MoE	Fused_moe	vLLM[d]
GEMM (quantized)	Gemm_quantize	StackAV[e]

[a] Triton language repository (reference kernels) [25].
[b] TorchInductor-generated Triton kernels from PyTorch [31].
[c] FlashAttention v2 Triton kernels from Dao-AILab [8].
[d] Fused MoE Triton kernels from vLLM [20].
[e] Quantized GEMM Triton kernels from StackAV [41].

Tuning Configurations. Tuning hyperparameters for Triton kernels on GPUs are reported in Tables 3, 4, and 5. On the V100 GPU, the total number of configurations (Total_Configs) is determined by all combinations of tiling settings (e.g., BLOCK_M, BLOCK_N, BLOCK_K, GROUP_M) and the number of wraps (num_wraps). On the A100 and H100 GPUs, the number of compilation (optimization) stages for software-pipelining loops is included, resulting in a large number of configurations. On the MI250 GPU, the configuration space also includes waves_per_eu, a hyperparameter that controls the use of vector general-purpose registers to achieve the desired occupancy levels. Consequently, the space is much larger than others. For instance, for attention kernels, the numbers of configurations on V100, A100, and MI250 are 96, 480, and 3200, respectively.

Settings. To train and evaluate the performance predictor, the collected dataset is randomly divided into 80% for training and 20% for

Table 3: Kernel config space on V100.

op name	BLOCK_M	BLOCK_N	BLOCK_K	GROUP_M	num_warps	Total_Configs
MM, Gemm_quantize, BMM	16, 32, 64, 128	16, 32, 64, 128	16, 32, 64, 128	1, 2, 4, 8	1, 2, 4, 8, 16, 32	1536
CONV	16, 32, 64, 128	16, 32, 64, 128	16, 32, 64, 128	1, 2, 4, 8	1, 2, 4, 8, 16, 32	384
Flash_attn	16, 32, 64, 128	–	16, 32, 64, 128	1, 2, 4, 8	1, 2, 4, 8, 16, 32	96
ReLU, Softmax, GeLU, Dropout, BN, LN, SiLU	16, 32, 64, 128, 256, 512, 1024	–	–	–	1, 2, 4, 8, 16, 32	42

Table 4: Kernel config space on A100/H100.

op name	BLOCK_M	BLOCK_N	BLOCK_K	GROUP_M	num_warps	num_stages	Total_Configs
MM, Gemm_quantize, BMM, Fused_moe	16, 32, 64, 128	16, 32, 64, 128	16, 32, 64, 128	1, 2, 4, 8	1, 2, 4, 8, 16, 32	1, 2, 3, 4, 5	7680
CONV	16, 32, 64, 128	16, 32, 64, 128	16, 32, 64, 128	1, 2, 4, 8	1, 2, 4, 8, 16, 32	1, 2, 3, 4, 5	1920
Flash_attn	16, 32, 64, 128	–	16, 32, 64, 128	1, 2, 4, 8	1, 2, 4, 8, 16, 32	1, 2, 3, 4, 5	480
ReLU, Softmax, GeLU, Dropout, BN, LN, SiLU	16, 32, 64, 128, 256, 512, 1024	–	–	–	1, 2, 4, 8, 16, 32	1, 2, 3, 4, 5	96

Table 5: Kernel config space on MI250.

op name	BLOCK_M	BLOCK_N	BLOCK_K	GROUP_M	num_warps	num_stages	waves_per_eu	Total_Configs
MM, Gemm_quantize, BMM, Fused_moe	16, 32, 64, 128	16, 32, 64, 128	16, 32, 64, 128	1, 2, 4, 8	1, 2, 4, 8, 16	1, 2, 3, 4, 5	1, 2, 3, 4, 5, 6	38400
CONV	16, 32, 64, 128	16, 32, 64, 128	16, 32, 64, 128	1, 2, 4, 8	1, 2, 4, 8, 16	1, 2, 3, 4, 5	1, 2, 3, 4, 5, 6	12800
Flash_attn	16, 32, 64, 128	–	16, 32, 64, 128	1, 2, 4, 8	1, 2, 4, 8, 16	1, 2, 3, 4, 5	1, 2, 3, 4, 5, 6	3200
ReLU, Softmax, GeLU, Dropout, BN, LN, SiLU	16, 32, 64, 128, 256, 512, 1024	–	–	–	1, 2, 4, 8, 16	1, 2, 3, 4, 5	1, 2, 3, 4, 5, 6	1400

testing. Each operator has its own predictor based on XGBoost [27]. A base model is initialized with 500 estimators. Empirically, the depth is set to 10, and the learning rate is 0.02. The XGBoost objective function is set to `reg:gamma` to ensure that all predicted values, e.g., latency, remain positive. Following prior work [36], we evaluate prediction accuracy using the mean absolute percentage error (MAPE).

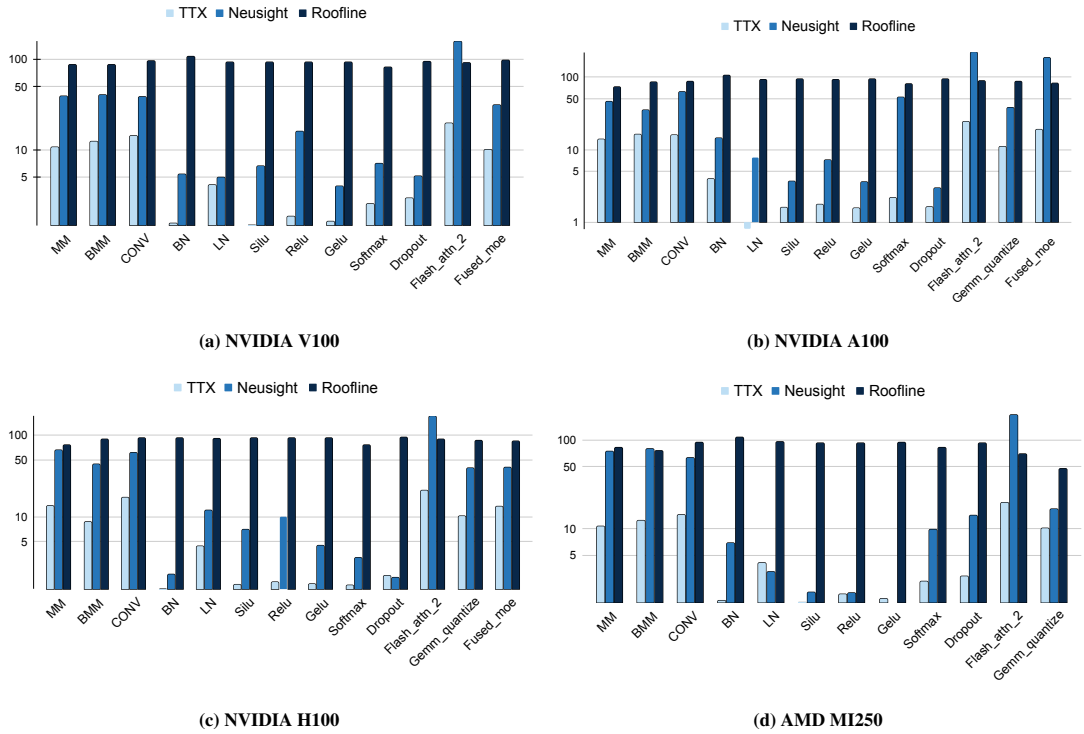
$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{1}$$

where y_i denotes the ground truth value and $\hat{y}_i$ represents the predicted value.

To evaluate autotuning performance, we use the set of input shapes from previous work [42] and collect data for all configurations of each shape to form the ground truth. Meanwhile, TTX generates a small set of tuning configurations (e.g., Top-k) based on its performance model. Given an input shape, the best-performing candidate among the top-k is normalized to match the best one obtained from the ground truth.

Baselines. Our latency predictor is compared against recent GPU kernel performance models, including Neusight [36] and the Roofline model [43]. The comparison is conducted using our Triton kernel dataset, with aligned model settings and, where applicable, feature representations. When training and evaluating the Neusight MLP model on our Triton kernel dataset, we exclude hardware-level features (e.g., L2 cache size and the number of SMs) to ensure a fair and consistent comparison. Meanwhile, Roofline relies on analytical performance bounds derived from peak hardware characteristics, including peak memory bandwidth and peak FLOPs. Kernel performance is estimated based on arithmetic intensity, defined as the ratio of total floating-point operations to the volume of memory transactions.

Cross-Kernel Setting. In addition to the per-kernel evaluation above, we study how well a *single* unified predictor generalizes across kernels. The unified variant trains one XGBoost across all kernels using the same hyperparameters as TTX, augmented with a fixed-length PTX-IR embedding produced by a small Transformer encoder pretrained with a contrastive (NT-Xent) objective on the same kernel corpus ($d_{\text{model}} = 128$, two layers, four heads, output dimension 128, temperature 0.1, 50 epochs). The encoder is frozen and applied per (kernel, configuration) prior to XGBoost training. We evaluate this variant under leave-one-kernel-out (LOKO): for each held-out kernel, the unified model is trained on the remaining kernels and tested on the held-out one. We report MAPE per kernel and the geometric mean across folds. This study uses V100 across



(a) NVIDIA V100

(b) NVIDIA A100

(c) NVIDIA H100

(d) AMD MI250

Figure 7: Latency prediction accuracy (MAPE, %, lower is better) across Triton operators on NVIDIA V100/A100/H100 and AMD MI250, comparing TTX with NeuSight and the Roofline model.

nine kernels (MM, BMM, LN, Softmax, BN, GeLU, SiLU, ReLU, Dropout); preliminary multi-seed and multi-GPU validation are ongoing.

4.2 Experimental Results

4.2.1 Results for Performance model. Figure 7 presents the prediction accuracy of the operations on Nvidia V100, A100, and H100, and AMD MI250. Specifically, complex operators such as GEMM, BMM, Convolution, FlashAttention, Quantized GEMM, and MoE exhibit MAPEs ranging from 9% to 22%. In contrast, the remaining seven lighter kernels achieve significantly lower error rates, ranging from 0.5% to 5%. This disparity can be attributed to the complex runtime behaviors of the compute-intensive operators, whereas the lighter kernels are influenced by less tuning parameters and exhibit more predictable performance patterns. This behavior is consistent across all the hardwares in our experiments.

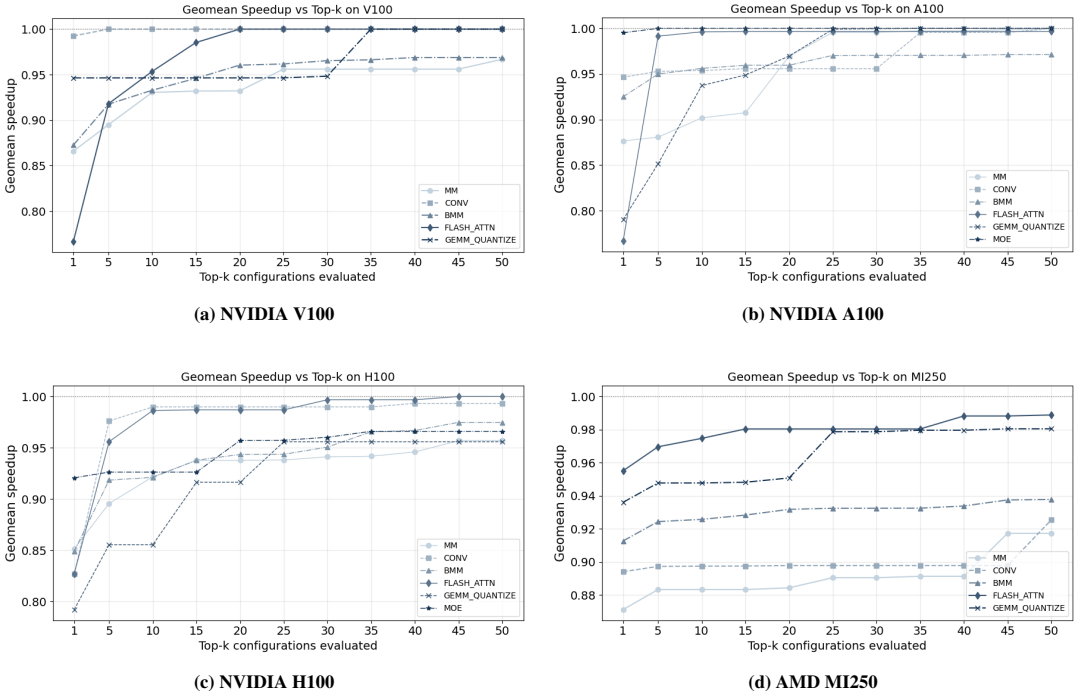
Although NeuSight performed well on cuDNN workloads, it consistently ranks second in our evaluation on the Triton dataset. We observed a distinct error pattern where NeuSight struggles with compute-intensive operators, exhibiting high MAPEs ranging from 19% to 219%. In contrast, for lightweight operators, the error rates are significantly lower (1.86% to 14%). These results suggest that while NeuSight performs well in its native cuDNN environment, it fails to generalize effectively to the Triton compiler context.

The Roofline model consistently exhibits poor predictive accuracy, often yielding a MAPE of approximately 100%. This limitation stems from its oversimplified assumptions, which fail to capture the complex, non-linear runtime behaviors inherent in GPU kernels. Consequently, the Roofline model consistently ranks third in our evaluation.

4.2.2 Cross-Kernel Generalization Analysis. A natural question is whether a single cross-kernel model could replace the per-kernel ensemble in TTX. We investigated this empirically using the unified variant described in Section 3.2. Under leave-one-kernel-out evaluation on V100 across nine kernels, the unified model achieves a geometric-mean MAPE of **37.7%** — substantially better than the Roofline baseline (∼100%) but markedly worse than per-kernel TTX (∼10% on heavy kernels, ≤5% on lightweight kernels).

The error decomposes asymmetrically by kernel family, as summarized in Table 6. Pointwise activations and dropout transfer well across the kernel boundary (4–25% MAPE), and the softmax / normalization folds remain in a usable range (∼32–53% MAPE). In contrast, the compute-bound matrix-multiplication folds remain stuck at ∼83% (MM) and ∼99% (BMM) regardless of encoder configuration. The unified model cannot extrapolate to the held-out kernel's shape × configuration interactions without direct training samples for that operator: GEMM-style kernels exhibit shape-dependent kernel selection (the Triton autotuner chooses qualitatively different code paths at different shapes), and a unified XGBoost trained without seeing any sample of the held-out kernel cannot reconstruct that surface from a frozen IR embedding alone.

This asymmetry empirically motivates the per-kernel design adopted throughout TTX. For pointwise and normalization kernels, a unified



(a) NVIDIA V100 **(b) NVIDIA A100**

(c) NVIDIA H100 **(d) AMD MI250**

Figure 8: Geometric-mean speedup as k increases (top-k configurations) across four GPUs.

Table 6: Cross-kernel LOKO MAPE on V100, broken down by kernel family. Compute-bound: MM, BMM. Pointwise: GeLU, SiLU, ReLU, Dropout. Normalization: LN.

Kernel family	TTX	Unified LOKO
Compute-bound	9–13%	83–99%
Softmax	1–3%	∼32%
Normalization	1–3%	∼53%
Pointwise	1–5%	4–25%
Geomean (9 folds)		**37.7%**

predictor is adequate for ranking; for the heavy compute-bound kernels that dominate ML workloads, per-kernel modeling is necessary to obtain the absolute-latency accuracy required for the top-50 selection. Closing the cross-kernel gap with hybrid IR-encoder predictors and few-shot per-kernel adaptation is the focus of follow-up work.[2]

4.2.3 Effectiveness of Applying the Model to Auto-Tuning. We evaluated six compute-intensive operators, each implemented with four distinct input shapes as detailed below. These workloads were executed utilizing the configuration spaces defined in Tables 3, 4, and 5. As illustrated in Figure 8, our method demonstrates robust performance: even at top_k=1, all kernels across all evaluated GPUs achieve at least 75% of the optimal speedup. Furthermore, with top_k = 50, the model-guided configurations consistently achieve over 90% of the oracle performance across all evaluated kernels and GPU architectures. Consequently, we select a top_k value of 50 for our framework to achieve an optimal balance between measurement overhead and performance gain.

While the search process is highly efficient on V100, A100, and H100, quickly locating the best configurations, the AMD MI250 presents a unique optimization challenge. Due to its massively expanded configuration space, the probability of the optimal configuration appearing in the top-ranked candidates is naturally lower compared to NVIDIA GPUs. Consequently, finding the absolute best performance on the MI250 necessitates exploring a broader range of candidates (a larger top_k). This distinction underscores the impact of hardware-specific search space characteristics on the efficacy of auto-tuning models.

[2]The LOKO study uses a single seed (42), runs on V100, and excludes CONV, FlashAttention, GEMM-quant, and fused MoE due to incomplete IR coverage in the unified-corpus pipeline; the BN fold is omitted from the family breakdown because its latency distribution under LOKO produces a pathological residual that we attribute to out-of-range training values rather than to encoder capacity.

Table 7: Normalized performance (%, higher is better) of the best configuration found by TTX vs. TorchInductor (max-autotune) for MM and BMM on NVIDIA V100 and A100. Scores are normalized to the oracle best configuration (entire config space) $\times 100$.

Op	Input shape	V100		A100	
		TTX	Inductor	TTX	Inductor
MM	(128,128,128)	100.00	55.84	76.60	67.97
	(32,512,512)	90.31	55.68	92.93	72.11
	(512,512,512)	97.06	96.49	95.26	78.50
	(1024,1024,1024)	86.07	96.64	100.00	91.62
BMM	(16,128,128,128)	90.64	98.52	99.33	94.52
	(16,32,512,512)	90.00	99.96	96.34	98.00
	(16,1024,1024,1024)	100.00	100.00	100.00	67.87

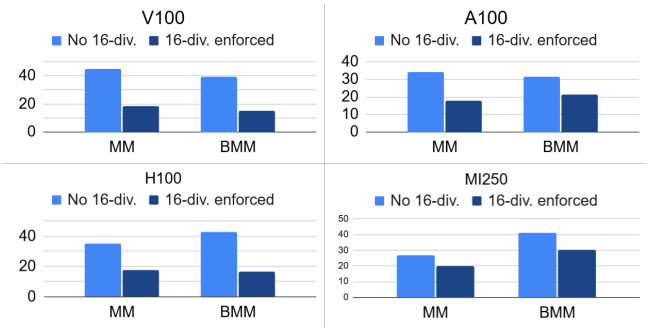


Figure 9: Impact of divisibility-by-16 features: Consistent MAPE reduction for MM/BMM over high-level feature sets.

4.2.4 Comparison with TorchInductor. TorchInductor's `max-autotune` benchmarks all valid configurations within a fixed candidate set (typically 15–20 options, depending on the device and input shape). To ensure a controlled comparison, we constrain TTX to evaluate the same number of candidates (Top-k) as Inductor for each case, yielding identical profiling overhead. Table 7 reports the resulting kernel performance, normalized to the oracle optimum (best configuration under exhaustive search).

With the same profiling budget, TTX consistently identifies faster configurations. For matrix multiplication (MM) operations on V100, Inductor's fixed candidate set misses good regions for smaller shapes (e.g., $(128, 128, 128)$ reaches 55.84% of oracle), while TTX attains the oracle solution (100%) and remains near-optimal on $(32, 512, 512)$ (90.31% vs. 55.68%). On A100, TTX improves MM by up to ∼21–29% over Inductor on medium shapes (e.g., $(32, 512, 512)$ and $(512, 512, 512)$). For BMM on A100, Inductor drops to 67.87% on the largest case $(16, 1024, 1024, 1024)$, whereas TTX finds the oracle configuration (100%). Overall, dynamic candidate selection from the full design space yields up to 44 percentage-point higher normalized performance without increasing profiling cost.

4.2.5 Ablation study. We evaluate the impact of divisibility-by-16 features on prediction accuracy. In the Triton compiler, fixed tuning configurations can still result in distinct PTX code paths based on whether input dimensions are multiples of 16. This compiler-level specialization leads to varied performance behaviors even for identical high-level parameters. To capture this effect, we compare Feature Set 1 (input shapes, tiling, and grid dimensions) against Feature Set 2, which incorporates additional binary divisibility indicators. Our results show that explicitly encoding this property resolves performance aliasing and significantly reduces MAPE across all architectures.

Results in Figure 9 demonstrate that incorporating divisibility-by-16 features consistently enhances model accuracy across all evaluated GPUs. Without these indicators, prediction errors remain high; for instance, the MAPE for MM and BMM on V100 drops significantly from 44.8% to 18.5% and 39.33% to 14.84%, respectively. This consistent improvement across A100, H100, and MI250 architectures stems from the model's enhanced ability to distinguish between Triton's specialized PTX code paths, which vary based on input dimension alignment. By explicitly encoding this property, the model effectively captures the performance fluctuations induced by compiler-level specialization.

Table 8: TTX MAPE: Using All IR Features vs. Excluding Non-critical Features.

Op	V100		H100	
	non-critical	All features	non-critical	All features
MM	15.69	10.67	13.68	12.76
BMM	14.39	12.34	9.03	8.70
CONV	16.10	14.34	18.47	17.31
Flash_attn	18.80	16.54	14.04	12.56
Fused_moe	10.50	9.98	13.31	10.15

In addition, we analyze the contribution of the eight IR features detailed in Table 1 and observe a clear hierarchy in their influence. Four primary features total instructions, arithmetic, special, and logic instructions emerge as the dominant drivers of model performance. In contrast, the remaining attributes (control, sync, and memory volumes) are found to be non-critical. As demonstrated in Table 8, excluding these four secondary features results in a negligible performance gap of less than 5%, confirming that the model's predictive power is concentrated within the core instruction-based metrics.

4.2.6 Feasibility Analysis of Static Configuration Subsets. Our methodology constructs a specialized performance model for each kernel to identify the optimal configuration for any given input shape. We explore the hypothesis that a small, representative subset of configurations might exist which consistently achieves at least 80% to 90% of the oracle performance across all input shapes. If such a subset existed, it would simplify the search process, especially in high-dimensional configuration spaces where finding the absolute optimum is computationally expensive. However, our experiments demonstrate that no such representative subset exists that can maintain high performance across diverse workloads. This finding confirms that a universal, input-independent configuration is insufficient, thereby justifying the necessity of our dynamic performance model to capture input-dependent variations.

Our analysis reveals that optimal kernel configurations vary substantially across different input shapes. Specifically, we evaluate configurations achieving at least 80% of the oracle performance (i.e., latency within 1.2× of the optimum).

As illustrated in Figure 10, while some configurations are shared between any two sets (e.g., 33 shared configurations for 128^3–512^3 and 2 for 512^3–1024^3), there is **zero intersection** among all three sets. This absence of a universal configuration subset across diverse workloads empirically proves that a static selection is insufficient, thereby justifying the necessity of TTX's input-aware automated tuning approach.

4.2.7 TTX Cost Analysis. The cost evaluation for TTX is presented in Table 9, which details the time required to collect 20,000 data points for each kernel across various GPUs. This collection time is influenced by several factors, including input shapes, tuning configurations, GPU specifications, and the specific kernel types. Notably, the training phase is highly efficient; using XGBoost on an AMD EPYC CPU, it takes only 3.00s, 3.45s, 3.14s, and 3.89s to train

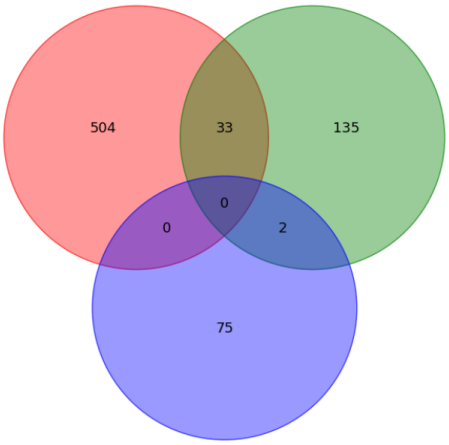


Figure 10: Overlap of configuration subsets achieving ≥80% of optimal performance across three input shapes for MM on NVIDIA A100: 128^3 (red), 512^3 (green), and 1024^3 (blue). The sparse intersection highlights the necessity for input-aware tuning.

Table 9: Data collection time (seconds) for 20k training samples per kernel on various GPUs.

Kernel	V100	A100	H100	MI250
MM	51 288	21 841	11 401	12 767
BMM	101 644	49 465	34 002	14 262
CONV	66 783	18 284	13 734	7159
Flash_attn	246 164	138 062	24 042	33 549
Fused_moe	53 233	148 967	12 480	x
Gemm_quantize	x	19 292	12 926	12 049
Other(Avg.)	3062	2934	2051	1390

Table 10: Time cost (miliseconds) for searching top_k configurations within the A100-specific configurations space. new

Kernel	TTX	Neusight	Roofline model
MM	2.12	4.45	2.32
BMM	1.23	3.23	2.36
CONV	1.67	2.67	2.69
Fused_moe	1.78	2.85	2.89
Flash_attn	2.12	3.56	2.29
Gemm_quantize	2.57	4.18	2.19
Other(Avg.)	1.49	1.67	2.18

on 20,000 data points for MM, BMM, CONV, and FLASH_ATTN, respectively.

By leveraging a lightweight XGBoost approach, TTX achieves superior performance in deriving top-k configurations. It delivers an average speedup of 1.87 over Neusight and 1.3 over the Roofline model. Notably, all three frameworks maintain low latency, completing tasks in just a few milliseconds (see details in Table 10).

5 Tool Release and Ecosystem Integration

We release TTX as an open-source toolkit under Apache 2.0,[3] comprising the predictor and feature extractors, pretrained artifacts for V100/A100/H100/MI250, the 13-kernel benchmark suite, and training scripts for retargeting to additional GPUs. Documentation and tutorials are hosted on a public site, and a Docker image reproduces all results in Section 4.

TTX integrates natively with Triton through the existing `prune_configs_by` interface, specifically the `perf_model` hook of `triton.autotune`. Adoption is a one-line change to an existing autotuned kernel:

```
@triton.autotune(
  configs=cfgs, key=["M","N","K"],
  prune_configs_by={
    "perf_model": ttx.Model("matmul","h100"),
    "top_k": 50,
  },
)
@triton.jit
def kernel(...): ...
```

No changes to user kernels or to upstream Triton are required; the integration uses an extension point that the Triton maintainers designed for exactly this purpose, ensuring forward compatibility with future Triton releases.

6 Conclusion

Beyond per-kernel accuracy, we further analyzed cross-kernel generalization with a unified predictor under leave-one-kernel-out evaluation. The analysis reveals a clean asymmetry — pointwise and normalization kernels transfer well across kernels, while compute-bound kernels (MM, BMM) do not — empirically justifying the per-kernel design adopted in TTX. Closing the cross-kernel gap on compute-bound kernels, particularly through hybrid IR-encoder predictors and few-shot per-kernel adaptation that would let TTX onboard previously unseen Triton kernels in minutes rather than GPU-hours, is the focus of ongoing work.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 770–778. [Online]. Available: https://doi.org/10.1109/CVPR.2016.90

[2] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[3] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 2019, pp. 6105–6114. [Online]. Available: http://proceedings.mlr.press/v97/tan19a.html

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.

[5] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[6] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale," *Advances in neural information processing systems*, vol. 35, pp. 30 318–30 332, 2022.

[7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] Anonymized for double-blind review; public URL provided in the camera-ready.

[8] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[9] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 18 332–18 346. [Online]. Available: https://proceedings.mlr.press/v162/rajbhandari22a.html

[10] C. Zhao, S. Zhou, L. Zhang, C. Deng, Z. Xu, Y. Liu, K. Yu, J. Li, and L. Zhao, "Deepep: an efficient expert-parallel communication library," https://github.com/deepseek-ai/DeepEP, 2025.

[11] K. Hong, G. Dai, J. Xu, Q. Mao, X. Li, J. Liu, K. Chen, Y. Dong, and Y. Wang, "Flashdecoding++: Faster large language model inference with asynchronization, flat GEMM optimization, and heuristics," in *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*, P. B. Gibbons, G. Pekhimenko, and C. D. Sa, Eds. mlsys.org, 2024. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/hash/5321b1dabcd2be188d796c21b733e8c7-Abstract-Conference.html

[12] NVIDIA, "Nvidia tesla v100 gpu architecture," 2017. [Online]. Available: https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

[13] ——, "Nvidia turing gpu architecture," 2018. [Online]. Available: https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf

[14] ——, "Nvidia a100 tensor core gpu architecture," 2020. [Online]. Available: https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf

[15] AMD, "Amd mi250 gpu architecture," 2021. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/instinct-business-docs/white-papers/amd-cdna2-white-paper.pdf

[16] K. M. Mhatre, V. G. P. Mulleti, C. J. Bansil, E. Taka, and A. Arora, "Performance analysis of gemm workloads on the amd versal platform," in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2025, pp. 150–161.

[17] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin *et al.*, "Tensorflow: Large-scale machine learning on heterogeneous distributed systems," *arXiv preprint arXiv:1603.04467*, 2016.

[18] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[19] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. K. Luk, B. Maher, Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, S. Zhang, M. Suo, P. Tillet, X. Zhao, E. Wang, K. Zhou, R. Zou, X. Wang, A. Mathews, W. Wen, G. Chanan, P. Wu, and S. Chintala, "Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 929–947. [Online]. Available: https://doi.org/10.1145/3620665.3640366

[20] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: https://doi.org/10.1145/3600006.3613165

[21] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: https://openreview.net/forum?id=VqkAKQibpq

[22] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze *et al.*, "{TVM}: An automated {End-to-End} optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 578–594.

[23] A. TVM, "Apache tvm," May 2023. [Online]. Available: https://tvm.apache.org/docs/

[24] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," *SIGPLAN Not.*, vol. 48, no. 6, p. 519–530, Jun. 2013. [Online]. Available: https://doi.org/10.1145/2499370.2462176

[25] P. Tillet, H. T. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, ser. MAPL 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 10–19. [Online]. Available: https://doi.org/10.1145/3315508.3329973

[26] R. Swann, M. Osama, X. Guo, B. Nelson, L. Zhang, A. Brown, Y. Ong, A. Yazdani, S. Siddens, G. Dasika, and A. Underwood, "tritonblas: Triton-based analytical approach for gemm kernel parameter selection," 2025. [Online]. Available: https://arxiv.org/abs/2512.04226

[27] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 785–794. [Online]. Available: https://doi.org/10.1145/2939672.2939785

[28] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016. [Online]. Available: https://arxiv.org/abs/1607.06450

[29] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *CoRR*, vol. abs/1502.03167, 2015. [Online]. Available: http://arxiv.org/abs/1502.03167

[30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6000–6010.

[31] PyTorch Contributors, "TorchInductor GPU Profiling," https://pytorch.org/docs/stable/torch.compiler_inductor_profiling.html, 2025, accessed: 2025-04-23.

[32] K. O'neal, P. Brisk, A. Abousamra, Z. Waters, and E. Shriver, "Gpu performance estimation using software rasterization and machine learning," *ACM Trans. Embed. Comput. Syst.*, vol. 16, no. 5s, Sep. 2017. [Online]. Available: https://doi.org/10.1145/3126557

[33] S. Hong and H. Kim, "An analytical model for a gpu architecture with memory-level and thread-level parallelism awareness," in *Proceedings of the 36th Annual International Symposium on Computer Architecture*, ser. ISCA '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 152–163. [Online]. Available: https://doi.org/10.1145/1555754.1555775

[34] ——, "An integrated gpu power and performance model," *SIGARCH Comput. Archit. News*, vol. 38, no. 3, p. 280–289, Jun. 2010. [Online]. Available: https://doi.org/10.1145/1816038.1815998

[35] G. Wu, J. L. Greathouse, A. Lyashevsky, N. Jayasena, and D. Chiou, "Gpgpu performance and power estimation using machine learning," in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*, 2015, pp. 564–576.

[36] S. Lee, A. Phanishayee, and D. Mahajan, "Forecasting gpu performance for deep learning training and inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 493–508. [Online]. Available: https://doi.org/10.1145/3669940.3707265

[37] Y. Li, Y. Sun, and A. Jog, "Path forward beyond simulators: Fast and accurate gpu execution time prediction for dnn workloads," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 380–394. [Online]. Available: https://doi.org/10.1145/3613424.3614277

[38] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás, "Gpu static modeling using ptx and deep structured learning," *IEEE Access*, vol. 7, pp. 159 150–159 161, 2019.

[39] L. Braun, S. Nikas, C. Song, V. Heuveline, and H. Fröning, "A simple model for portable and fast prediction of execution time and power consumption of gpu kernels," *ACM Trans. Archit. Code Optim.*, vol. 18, no. 1, Dec. 2021. [Online]. Available: https://doi.org/10.1145/3431731

[40] W. Wen, "Introduction to torch.compile," https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html, accessed: 2025-04-23.

[41] StackAV, "Conch, a "standard library" of triton kernels," gitHub repository. [Online]. Available: https://github.com/stackav-oss/conch

[42] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating High-Performance tensor programs for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 863–879. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/zheng

[43] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, p. 65–76, Apr. 2009. [Online]. Available: https://doi.org/10.1145/1498765.1498785